Range Queries

Kanishk Goel

1 June, 2026

1 Range Queries

The task is to calculate a value based on a subarray of an array. A naive way to answer a range query is to iterate over i to j such that $i < j$ and find the value that the query asks for. This runs in $O(n)$ and for q queries will require $O(nq)$ time. Notation - $func_q(a_1, a_2 \dots a_n)$

2 Sum Queries - Prefix Sum Subarray

Range query where value required is simply the sum of the elements in that range. This is solved in $O(n)$ via prefix sum array.

A variant of the same is to generalize the idea in higher dimensions. For example in $2D$, we can require the sum of a rectange from (i, j) to $(i + l, j + b)$.

The solution to this, is to simply calculate a prefix sum subarray f, which calculates sum of the rectangle from $(0, 0)$ to (i, j). Now the answer would simply be $f(i, j) + f(i + l, j + b) - f(i, j + b) - f(i + l, j)$.

3 Min Queries - Sparse Table

This is a $O(n \log n)$ preprocessing method, after which we can answer most queries in $O(\log n)$ time and any minimum/maximum query in $O(1)$ time.

3.1 Idea

Every number has a binary representation, therefore precalculate all values of $min_q(a, b)$ where $b - a + 1$ is a power of 2.

Every 2^k range can be calculated by using the previous 2^{k-1} ranges as

$$min_q(a, b) = \min(min_q(a, a + w - 1), min_q(a + w, b)),$$

where $w = (b - a + 1)/2$

Now any query (a, b) can be solved by representing the range $[a, b]$ as $[a, a+k-1] \cup [b-k+1, b]$ where k is the largest power of 2 that does not exceed $b - a + 1$.

$$min_q(a, b) = \min(min_q(a, a + k - 1), min_q(b - k + 1, b)),$$

3.2 Implementation

We use a $2D$ array, where $st[i][j]$ stores the answer for the range $[j, j + 2^i - 1]$. Size of the array is $(K + 1) \times MAXN$ where $MAXN$ is the biggest possible array length and $K \geq \lfloor log_2 MAXN \rfloor$. $MAXN$ dimension is second for cache friendliness.

3.3 Conclusion

Sparse Table is therefore, best used for idempotent functions (functions whose repeated application doesn't change the result beyond the first time).

4 Fenwick Tree

Consider f to be some group operation (binary associative function over a set with an identity element and inverse elements) and A be an array of length n. Denoting infix notation of f as $*$ i.e $f(x, y) = x * y$ for arbitrary integers x, y, the Fenwick Tree data structure:

- Calculates value of f over interval $[l, r]$ in $O(log\, n)$ time.
- Updates the value of an element of A in $O(log\, n)$ time.
- Requires $O(n)$ memory

The Fenwick Tree is just an array $T[0, \ldots n - 1]$, where each element is

$$T_i = \sum_{j=g(i)}^{i} A_j$$

for binary operation f to be addition.

4.1 What is $g(i)$

$g(i)$ replaces all the trailing 1s in the binary representation of i, equivalent to $i \,\&\, (i + 1)$.

4.2 f over $[0, i]$

we have to start from i and go to 0, we know we have values stored for $[g(i), i]$ and $[g(g(i) - 1), g(i) - 1]$ and so on.

4.3 Update

To update $A[i]$, we need to know all the $T[j]$s such that $g(j) \leq i \leq j$. Notice, all the js $< n$ which are made from setting unset bit of i (and all subsequent i'), trivially have $j \geq i$ and $g(j) \leq i$. Therefore, we define $h(j) = j|(j + 1)$ and use this continually until $j' >\geq n$ (because 0 based indexing).

4.4 One based indexing

$$g(i) = i - (i \,\&\, -i) \wedge h(i) = i + (i \,\&\, -i)$$

$g(i)$ simply becomes toggling of the last set bit in the number.

4.5 Point Update and Range Query

Normal Fenwick Tree as described above.

4.6 Range Update and Point Query

We wish to increment range $[l, r]$ by x and know the value of $A[i]$. We do this by storing the difference array d. $\forall i,\, i > 0, d[i] = A[i] - A[i - 1]$ for 0 based indexing and $d[0] = A[0]$.

4.6.1 Update

Update simply becomes adding x to $d[l]$ and substracting x from $d[r + 1]$ using the Fenwick point update method. To get $A[i]$, we simply obtain the prefix sum $P(i)$ on the Fenwick Tree using the Fenwick range query method.

4.7 Range Update and Range Query

We maintain two fenwick trees now, B_1 and B_2. For a range $[l, r]$, we require

$$\sum_{k=m}^{i} a_k = \sum_{k=1}^{i} a_k - \sum_{k=1}^{m-1} a_k \tag{1}$$

$$a_k = \sum_{j=1}^{k} d_j \tag{2}$$

Using 1 and 2 (remember $[l, r]$ is the range in which update was done, $P(i)$ is

the prefix sum for), we get

$$P(i) = \sum_{k=1}^{i} a_k$$

$$P(i) = \sum_{k=1}^{i} \sum_{j=1}^{k} d_k$$

d_j contributes in $d_j, d_{j+1}, \ldots d_i$ i.e equal to $i - j + 1$. therefore,

$$P(i) = \sum_{j=1}^{i} d_j \cdot (i + 1 - j)$$

Therefore, we split into Fenwick Trees, B_1 stores d_j and B_2 stores $d_j \cdot j$

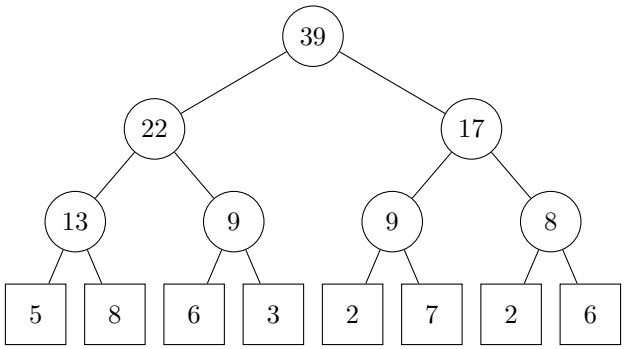
$$P(i) = (i + 1) \cdot query(B_1, i) - query(B_2, i)$$

5 Segment Tree

Binary tree such that the nodes on the bottom level of the tree correspond to the array elements and other nodes contain information needed for processing range queries.

0	1	2	3	4	5	6	7
5	8	6	3	2	7	2	6

will have the segment tree:



5.1 Range Query

For sum operation any range $[a, b]$ can be calculated as $O(log\, n)$ number of internal nodes (or using root node). We start from root node and recursively branch out to their children. We will only have 4 pointers parallelly in a tree's level, justifying the runtime (can be shown rigorously via induction).

Reason: Most ranges are fully contained in the range and only 2 (one with l contained in between and one with r in between) are the only ranges that recurse (at each level).

5.2 Update

For any array node to be updated, just update it and go bottom up to the root following the path and updating the sum. For example, here for updating 7, we take the route $7 - 9 - 17 - 39$. Hence, $O(log\, n)$ number of operations are required for updation.

5.3 Implementation

When the length n is not a power of 2, then the whole array does not need to be filled. Space required $= 2^{\lceil log_2\, n \rceil + 1} < 4n$.

Segment tree requires 2 points:

1. The value to store at each node of the segment tree (eg. a sum segment tree stores sum of elements in range $[l, r]$).
2. The merge operation that merges two siblings in a segment tree (eg. in a sum segment tree, nodes for $[l_1, r_1]$ and $[l_2, r_2]$ merge to node of range $[l_1, r_2]$).

For memory efficient storage, Euler Tours are used. The subtree from a node x is equivalent to the contigous segment starting from x. Hence, its use to reduce memory to $2n$. The left child of v is at $v + 1$ and right child is after the left child's subtree.